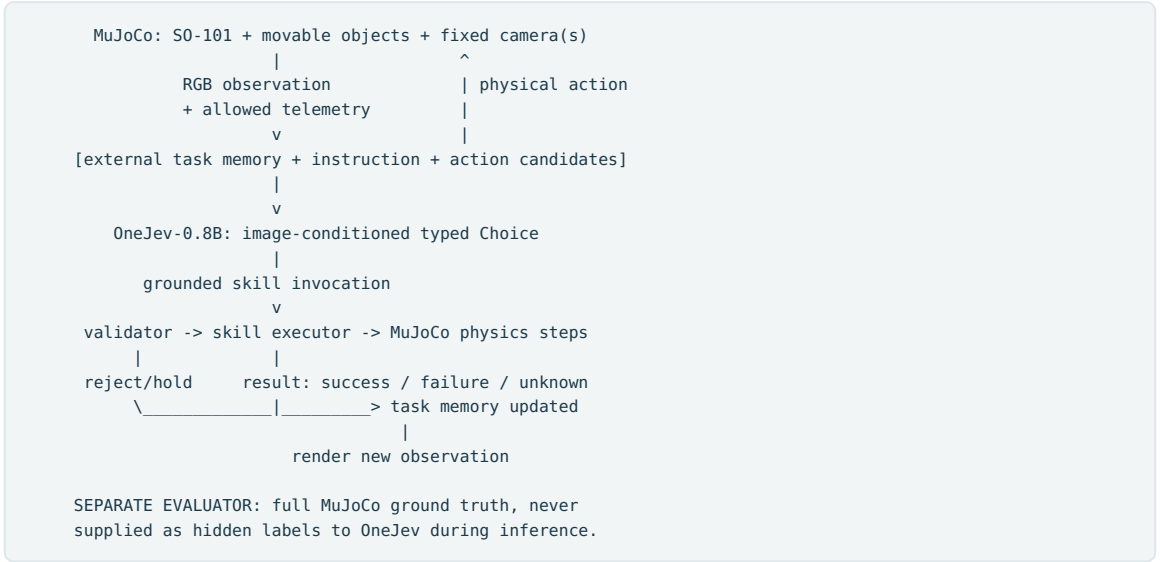
Simulation-First Visual Skill Agent — Final Implementation Plan

Version: 1.0 · **Decision date:** 16 October 2026
Hardware: MacBook Air M5, 16 GB unified memory · **Mode:** simulation only
Primary outcome: prove that a compact multimodal decision model can compose and verify reusable manipulation skills on held-out simulated tasks.

> **Decision:** Use **MuJoCo (native Python) + the Menagerie SO-101 model + OneJev-0.8B (GGUF/llama.cpp) + scripted closed-loop skill controllers + an external task-state manager + a separate evaluator**. Do **not** train or run a learned world model, WAM, OpenPI, GR00T, Isaac Sim, or a custom vision encoder in the initial phase. This is an implementation target, **not a claim that the combined stack has already been validated on an M5 Air**. [1–4]

0. Scope and architecture



Two clocks: the selected skill executes with local feedback at the simulator/controller timestep; the visual selector is called at the beginning, completion, failure, or uncertainty of a skill. **OneJev does**

not output joint trajectories. In Phase 1 the skills are **scripted**, not pretrained. Later, individual skills may be replaced by learned action policies. [3, 5, 6]

Non-goals now: real motor drivers; real camera calibration; physically faithful sim-to-real; arbitrary dexterity; RL from scratch; a learned world dynamics model; text captions between the camera and OneJev; video-input serving; a second VLM.

Terminology: `ground_truth` means privileged simulator state. `observation` means what the model can legitimately see (image, provided instruction, robot readings, documented execution feedback). A *valid selected skill* is not yet a *successfully executed skill*.

1. Required software and downloads — macOS setup (Phase 0)

1.1 Install now versus later

Status	Component	Exact purpose
NOW	Apple command-line tools, Homebrew, Git	Build environment and versioned source downloads
NOW	Python 3.12 + isolated virtual environment	Single reproducible Python runtime for simulation and client
NOW	`mujoco`, `numpy`, `pillow`, `pytest`, `pyyaml`	Physics, images, tests and configuration
NOW	MuJoCo Menagerie `robotstudio_so101`	Official SO-101 MJCF, meshes, existing box scene
NOW	OneJev source (`qev`) + Homebrew `llama.cpp`	Image-capable typed-decision client/server on Apple Silicon
NOW	OneJev-0.8B GGUF `Q8_0` + multimodal projector	First local model; downloaded on first server launch
LATER	`mink`	Differential IK and collision-aware configuration checks, if basic joint control is insufficient
LATER	`lerobot[training]`, PyTorch MPS	Supervised imitation learning / ACT; install in a separate environment
NOT NOW	robosuite, LIBERO, Isaac Sim, MJX/ Playground	Stronger tasks or GPU-batched training only after local gates

Why Q8_0: the 0.8B GGUF is approximately 1.08 GB, plus a separate image projector (approximately 116 MB for its Q8 projector). The 4B OneJev BF16 model is roughly 10.4 GB *before* runtime overhead and is not the starting choice on 16 GB unified memory. Quantization may change decision probabilities: evaluate the Q8 model as its own policy. [3, 4]

Real hardware not sim

1.2 Commands — run in Terminal on the Mac

Homebrew must be available from brew.sh. Install Xcode Command Line Tools **only if not already installed**:

```
xcode-select -p || xcode-select --install
brew install python@3.12 git llama.cpp
uname -m                        # must print arm64
python3.12 --version
command -v llama-server        # must resolve after Homebrew install
```

Create the project and Python environment:

```
mkdir -p ~/visual-skill-agent
cd ~/visual-skill-agent
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install 'mujoco>=3.1.3,<4' numpy pillow pytest pyyaml
mkdir -p vendor models config scenes src tests data/raw data/episodes results
```

Download **only the SO-101 Menagerie assets**, not the whole gallery. Git sparse checkout requires a recent Git (Homebrew Git suffices):

```
git clone --depth 1 --filter=blob:none --sparse \
  https://github.com/google-deepmind/mujoco_menagerie.git \
  vendor/mujoco_menagerie
git -C vendor/mujoco_menagerie sparse-checkout set robotstudio_so101
ls vendor/mujoco_menagerie/robotstudio_so101/scene_box.xml
```

Download OneJev and install **its non-PyTorch GGUF client path**:

```
git clone --depth 1 https://github.com/OmniJev/OneJev.git vendor/OneJev
python -m pip install -e vendor/OneJev
qev --help
```

Pin the actual installed sources **after** the smoke tests:

```
git -C vendor/mujoco_menagerie rev-parse HEAD > config/menagerie.commit
git -C vendor/OneJev rev-parse HEAD > config/onejev.commit
python -m pip freeze > config/python.freeze.txt
```

Do **not** mix the older `mujoco-py` package into this environment. On macOS, use `mjpython` when running a program with a **passive interactive viewer**; the official MuJoCo docs require it for that viewer mode. [1, 2]

1.3 Download and inspect the first world

`scene_box.xml` is an official **robot + ground plane + free-moving green box** example. **It does not contain a bowl, two-camera task, or a completed pick-and-place controller.** We will add those later. [2]

```
python -m mujoco.viewer \
  --mjcf=vendor/mujoco_menagerie/robotstudio_so101/scene_box.xml
```

Next check that the XML compiles and inspect the actual actuator ordering; **never assume the six control indices correspond to a guessed joint order**:

```
python - <<'PY'
import mujoco
p = 'vendor/mujoco_menagerie/robotstudio_so101/scene_box.xml'
m = mujoco.MjModel.from_xml_path(p)
d = mujoco.MjData(m)
mujoco.mj_forward(m, d)
print('MuJoCo', mujoco.__version__, '| nq', m.nq, '| nv', m.nv, '| nu', m.nu)
for i in range(m.nu):
    print(i, mujoco.mj_id2name(m, mujoco.mjtObj.mjOBJ_ACTUATOR, i),
          m.actuator_ctrlrange[i].tolist())
PY
```

Create and run a **complete image smoke test** (run in a normal logged-in macOS graphical session):

```
cat > src/render_smoke.py <<'PY'
import mujoco
from PIL import Image

model = mujoco.MjModel.from_xml_path(
    'vendor/mujoco_menagerie/robotstudio_so101/scene_box.xml')
data = mujoco.MjData(model)
mujoco.mj_forward(model, data)
renderer = mujoco.Renderer(model, height=480, width=640)
renderer.update_scene(data)
Image.fromarray(renderer.render()).save('data/raw/observation.png')
renderer.close()
print('saved data/raw/observation.png')
PY
python src/render_smoke.py
open data/raw/observation.png
```

Check that the frame actually shows the arm and box; a screenshot with no useful target is **not** a passing observation. For a passive *interactive* viewer inside your own program, use `mjpython` on macOS. [1]

1.4 OneJev image-inference smoke test (second Terminal)

From the same project and activated environment:

```
qev serve --gguf mradermacher/OneJev-0.8B-GGUF:Q8_0 --port 8000
```

This is the **documented OneJev GGUF serving path**, with the 0.8B model substituted for its 4B example. It should retrieve the GGUF model and appropriate image projector through the backend's normal download path. The image-capable **`qev` System One interface** is essential: a generic text-chat endpoint is *not* a substitute. If this route fails on your installed `llama.cpp` version, stop and record the error; do not silently switch to text-only decisions. [3, 4]

With the server still running, save the following as `src/smoke_onejev.py` and run `python src/smoke_onejev.py` from the activated project environment in a third Terminal:

```
from qev import Client, Choice
from qev.media import data_uri

response = Client('http://localhost:8000').system_one(
    state={
        'instruction': 'Approach the visible green box',
        'screen': '<image:1>',
        'gripper': 'open',
        'previous_skill_result': 'none',
    },
    media=[{'type': 'image', 'data': data_uri('data/raw/observation.png')}],
    questions={
        'next': Choice(
            'Which action is most appropriate for the visible scene?',
            {
                'approach_box': 'Move the gripper towards the visible box',
                'observe': 'Observe again because evidence is inadequate',
                'stop': 'Do not move',
            },
        ),
    },
)
print(response.answers['next'].probabilities)
```

Phase 0 exit: viewer opens, model compiles, an image is saved and visually checked, and OneJev returns a probability for every named option. Also record peak memory in Activity Monitor. **No robotics claim is made yet.**

---

2. Build sequence with required deliverables and gates

Phase 1 — A fully scripted working simulator (NO OneJev decisions)

Purpose: make the underlying physical environment and skill controllers work *before* asking whether the model plans well.

Implement, in this order:

1. Copy the base MJCF into your project's `scenes/` workflow or generate a derived scene; do not modify vendor files. Start from the provided green cube, but **do not assume its default world position (x = 0.5 m) is reachable**. Move it into a verified reachable region in your derived scene. Add a fixed overhead camera. **One camera first**; enable a second view only after establishing the single-view baseline.
2. Inspect actuator names, joint limits, gripper actuation, joint conventions and available camera sites. Implement position-controlled motion with bounded setpoints. Use `mink` only if target-pose IK is actually needed. [2, 7]
3. Implement a known reachable pregrasp target and the sequence `approach → grasp → lift → move/place → release → retract → verify`

as **scripted controllers**. During execution, call MuJoCo physics `mj_step` at the configured timestep; monitor contacts and gripper status.

4. Add a simple target zone/tray. The downloaded example contains a box **object**, not a receiving bowl. For the first test, a valid *placement region* is enough; use an open-top receptacle later if you want a genuinely physical containment test.
5. Define success using simulator-measured object position, release and settling, rather than simply checking that a command was sent.

Deliverables: `scenes/pick_place.xml`, `src/controllers.py`, `src/skills.py`, `src/task_env.py`, `tests/test_controllers.py`, a 30-second recorded episode, and a repeatable `python -m ...` entry point that you implement in the repository.

Tests / gate: zero invalid actuator commands; a successful lift physically retains the object; no object teleportation; after release the target remains in the goal region during a settle period; **≥9/10 successful scripted episodes on 10 fixed, explicitly reachable initial states**. If this fails, fix physics/controllers—not OneJev.

Phase 2 — Versioned skill library, observations and independent state

Purpose: make every action a callable, verifiable skill rather than a vague model phrase.

Define `SkillInvocation = {skill_id, target_id, destination_id?, parameters, observation_id, state_version, deadline}` and `SkillResult = {status: success|failed|unknown|cancelled, evidence, event_id}`. Never accept free-form executable Python from the model.

Skill	Minimum precondition	Execution result to verify
`approach(object)`	object selected; proposed target reachable	end effector near pregrasp pose
`grasp(object)`	alignment valid; gripper available	retained object, not merely `close` commanded
`lift(object)`	grasp evidence sufficient	object lifted above support, still retained
`place(object, zone)`	held object; reachable destination	object moved to release zone
`release(object)`	above permitted region	gripper opened; object no longer held
`retract()`	clearance trajectory available	gripper safely clear of object
`verify(goal)`	new observation available	goal confirmed / failed / unknown

State ownership: MuJoCo's `MjData` is the physical truth. An **external task manager** owns the goal, active skill, prior actions, observed outcomes, retry count and state version. The selector sees **rendered observation + permitted telemetry + causal event history**, *not*

privileged target coordinates or evaluator labels. The controller/validator may use simulator state for low-level feasibility and safety; log that use explicitly so visual understanding is not overstated.

Tests / gate: each skill has positive, negative and timeout tests; invalid invocation is rejected without changing physical state; `grasp commanded` cannot be misreported as `grasp succeeded`; state versions monotonically increase; reset with a given seed reproduces the scene.

Phase 3 — OneJev in the closed loop (NO training)

1. Create `src/selector_onejev.py`: accepts `ObservationPacket` (one image, telemetry, goal, latest event) and **complete grounded candidate invocations**, returns Choice probabilities.
2. Create `src/validator.py`: validates syntax, target existence, freshness, preconditions, reachability and collision constraints. Invalid choices **must never be executed**, regardless of probability.
3. Request decisions at skill boundaries, failure or uncertainty. The selected skill runs under its own controller until it completes, fails or times out.
4. When a grasp fails or the object slips, refresh observations and let OneJev choose among `reobserve`, `approach`, `grasp`, `recover` or `stop`, subject to guards. Never continue automatically to `lift` after an unverified grasp.
5. Use isolated Choice/Noul questions only for independently answerable issues. Two parallel questions cannot depend on one another's newly selected answers; include the **whole skill+target** combination in one Choice when coupling matters. [3]

Baseline arms: (A) scripted reference sequence and deterministic transitions, (B) OneJev from images, (C) oracle planner using privileged ground truth (optional until tasks are compositional). Keep the action library, controller, simulator and task seeds identical when comparing A and B. [5, 8, 9]

Tests / gate: over 20 known-reachable in-distribution tasks, target **≥16/20 verified completions** as an *internal go/no-go target*, **100% rejection of deliberately invalid invocations**, no state desynchronization, and a log for every decision and physical result. If the model underperforms, label failures as perception, grounding, sequencing, feasibility, execution, or verification—not just “model error”.

Phase 4 — Evaluation and generalization (freeze this before fine-tuning)

Ground-truth oracle uses simulator state to calculate successful task completion; it does not feed answers to OneJev. Use a fixed seed registry and independent train/dev/test groups. Do not put near-identical frames of the same episode across splits.

Suite	Perturbation / purpose	First evaluation budget
ID	Familiar objects, familiar placement patterns	20 held-out episodes
Spatial	Same objects, unseen reachable positions	20 episodes
Visual	Colour/material/lighting/distractors	20 episodes
Language	Paraphrases with identical goals	20 episodes
Composition	New valid sequences of familiar skills	20 episodes
Uncertainty	Occlusion, contradictory evidence, request-to-reobserve	10 targeted cases
Recovery	Scripted slip, bad grasp, invalid action proposal	10 targeted cases

Metrics, reported independently: ground-target selection accuracy; valid-invocation rate; physical skill success; *verified* episode success; extra/unnecessary actions; false-positive completion; recovery rate; per-suite generalization drop from ID; reported probabilities vs empirical correctness (Brier score / calibration bins); median/p95 end-to-end decision time and observation age (diagnostics, not the primary research objective).

Mandatory controls: a random valid-skill policy, a deterministic scripted planner/finite-state reference, and OneJev on the same test set. `Pyperplan` is **optional** for generating symbolic oracle plans only after task sequences become complex; it does not consume camera pixels. [8, 9]

Gate: ship a per-suite table and at least 10 replayable failure traces with exact seeds; no evaluator leakage; every claimed generalization gain is compared with the same controls and the same observation budget. **Do not fine-tune on the sealed test suites.**

---

3. Training and stronger simulation — only after Phases 0–4

Phase 5A — Fine-tune *the selector* if failures justify it

Trigger: Phase 4 demonstrates repeatable visual grounding or sequencing errors that are **not** explained by faulty controls, poor state inputs, or a broken skill contract.

Data: collect labeled `(image(s), allowed visible state, goal, candidate invocations, correct/acceptable decision set, observed outcome)` examples. Include near-misses, partial occlusion, slips, ambiguous evidence and examples where `observe/hold` is correct. Train/dev/test must split by episode and held-out combinations.

Method: begin with supervised decision training, using OneJev's published training code and its **cross-entropy + Brier** decision objective as the reference; its published recipe freezes the vision tower. Review and adjust the repo's actual 0.8B training config and dataset loader before any run. **Do not assume the released training scripts support efficient Apple MPS training on a 16 GB Air.** The documented training path is oriented to PyTorch/CUDA; plan a short rented NVIDIA session only if a memory probe demonstrates feasibility. [3]

Tests / gate: compare the fine-tuned checkpoint against the *same* frozen base-model Q8 evaluation; report improvement by suite, calibration, regression on old tasks, model size and reproducibility. Keep the original inference baseline archived. If there is no statistically meaningful improvement on held-out tasks, keep the pretrained model.

Phase 5B — Learn *individual motor skills* (a different training problem)

Trigger: deterministic skills work, but you need robustness to new object poses/contacts not handled by script.

Data: log successful scripted/teleoperated MuJoCo trajectories with synchronized camera frames, joint positions, *executed joint/action targets*, skill identifiers, goals and physical outcomes. Convert to a **LeRobotDataset-compatible** schema, including fixed action order, units, camera names, framerate and normalization statistics. [10]

Starting policy: ACT, not OpenPI or a large VLA. LeRobot documents ACT as its lighter imitation-learning entry point and supports the `mps` device on Apple Silicon; install `lerobot[training]` in a **separate virtual environment** and run a tiny batch + memory probe first. 16 GB is not a promise of workable training for every configuration. If necessary, train on rented GPU. [10, 11]

Tests / gate: each learned skill is evaluated alone on the same reachable seed sets and held-out object poses, then swapped into the *unchanged* Phase-3 skill interface. Do not credit OneJev for improvements due entirely to a stronger motor skill.

Phase 6 — Increase simulator/task difficulty in this order

1. **Same MuJoCo engine, harder scenes:** two camera views, more objects and clutter, randomized camera/lighting, changed friction/masses, partial occlusion, true receiving tray, failed grasps and out-of-reach targets. This is the first upgrade; no new engine required. Optionally add Menagerie's `i2rt_yam` model (`git -C vendor/mujoco_menagerie sparse-checkout add i2rt_yam`) to test skill-interface portability; it still needs a new robot-specific controller. Use NVIDIA's domain-randomization methods as a test-design reference, not its heavy software stack. [2, 12]
2. **robosuite** (optional): reuse MuJoCo with prebuilt manipulation tasks and controller references. Its task robots may differ from SO-101, so results are **not direct SO-101 transfer**. [13]
3. **LIBERO** (optional research benchmark): use when studying compositional/lifelong manipulation on standardized task suites. Its published installation path depends on older Python/CUDA/PyTorch packages; isolate on a compatible Linux machine rather than treating it as a frictionless M5 Air install. [14]
4. **MJX / MuJoCo Playground** (optional, accelerator-focused): only if large-scale batched RL or policy training becomes necessary; not needed for initial local testing. [15]
5. **Hardware transfer** (outside this document's active scope): only after matched robot model, camera, control, dataset conventions and safety tests exist.

Do not upgrade the simulator because a model makes incorrect visual decisions. Upgrade when the current environment cannot represent a failure mode or task you need to measure.

---

4. File layout, task checklist and release gates

```
visual-skill-agent/
  .venv/                          # local only, not committed
  vendor/{mujoco_menagerie,OneJev}/
  config/{menagerie.commit,onejev.commit,python.freeze.txt}
  scenes/{pick_place.xml,scenario_registry.yaml}
  src/{render_smoke.py,task_env.py,controllers.py,skills.py,
       state_manager.py,selector_onejev.py,validator.py,evaluate.py}
  tests/{test_sim.py,test_skills.py,test_validator.py,test_reset.py}
  data/{raw/,episodes/,splits/}
  results/{runs/,figures/,metrics.csv}
  README.md
```

Engineering sequence (do one row before starting the next):

Gate	Required artifact	Pass/stop rule
G0 — install	viewer, actuator inventory, PNG, OneJev Choice response	All four work locally; otherwise fix setup, not model
G1 — skills	one fully scripted physical pick-and-place	≥9/10 fixed, reachable scenes; no fake attachment/teleport

Gate	Required artifact	Pass/stop rule
G2 — contracts	deterministic state + invocation validator + tests	invalid actions never reach actuators; reset reproducible
G3 — integration	image → OneJev → skill → result closed loop	target ≥16/20 ID goal successes; all steps replayable
G4 — evaluation	120-case benchmark + baseline table	all suites measured; no leakage; 10 failure traces
G5 — selector SFT	optional trained checkpoint	only keep if out-of-sample gains beat frozen base
G6 — motor learning	optional ACT skill + standardized dataset	per-skill success and regression independently measured
G7 — richer sim	randomized MuJoCo → optional robosuite/ LIBERO	justified by missing evaluation coverage, not novelty

Record for each episode: source commit hashes; model/ quantization; scenario/seed; camera config; actual observed images and timestamps; visible input state; decision candidate set and probability distribution; chosen invocation; validator result; joint/action trajectory; skill events; **separate privileged ground truth**; terminal goal score. This allows precise failure attribution and prevents accidental claims that the model observed hidden simulator state.

If G0 is blocked on Mac: save the exact install/inference error and version information. Keep MuJoCo experiments moving without OneJev while resolving the GGUF backend; don't substitute a plain chat/VLM response for the decision-head API and present the results as equivalent.

---

5. Primary references, ranked by implementation importance

The rankings reflect **how directly the source informs this build**, not citation count or scientific importance. `[n]` markers above refer here. These are **original projects, official documentation, primary papers, or their maintainers**; exploratory claims from third-party posts are intentionally excluded from the implementation contract.

1. **MuJoCo (Google DeepMind):** Python API, installation, rendering and macOS viewer caveat. Physics and observation engine. Official repository.
2. **MuJoCo Menagerie:** Official repository / model-quality notes · SO-101 model and `scene_box.xml`. Exact starting robot/world asset; model requires MuJoCo ≥3.1.3.
3. **OneJev:** Official repository and API/training README · OneJev-0.8B model card. Image-conditioned typed decision model, frozen vision tower recipe and training loss.
4. **OneJev GGUF build:** mradermacher OneJev-0.8B GGUF. Q8/Q4 and separately packaged multimodal projector. llama.cpp.

5. **ARCH (CoRL 2025):** Hierarchical Hybrid Learning for Long-Horizon Contact-Rich Robotic Assembly. Main skill-hierarchy architectural precedent.
6. **SayCan (Google):** Project and research · paper. High-level skill usefulness versus grounded affordances/feasibility.
7. **Mink:** Official Python/MuJoCo inverse-kinematics repository · IK guide. Optional controller/IK implementation.
8. **VirtualHome:** Original research code. Explicit symbolic preconditions/effects and activity programs; conceptual interface reference, not the physics engine.
9. **Pyperplan:** Official repository. Optional symbolic oracle planning when task variety grows.
10. **Hugging Face LeRobot:** Main repository · dataset/policy documentation · Apple MPS training note. Dataset and skill-policy training.
11. **ACT:** Original Action Chunking Transformer paper · LeRobot ACT implementation · ACT docs. First learned motor-skill baseline.
12. **NVIDIA synthetic-data/domain randomization:** Official tutorial. Experimental perturbations, not a software dependency.
13. **robosuite:** Official repository · task docs. Next-level MuJoCo manipulation environments.
14. **LIBERO:** Original repository · paper. Later standardized task/transfer evaluation; older GPU-oriented dependencies.
15. **MuJoCo Playground:** Project · paper. Only for future batched/accelerator RL.
16. **SIMA (Google DeepMind):** Research overview · technical paper. Visual observation and instruction-following in interactive virtual worlds.
17. **Optional higher-capacity policy research:** SmolVLA paper · SmolVLA code · OpenVLA paper and code · OpenPI · Octo. Study only when changing action-policy capacity/representation.
18. **Recent hierarchical transfer research:** SkillNet (ICML 2026). Further reading after the functional baseline, not an implementation dependency.

First action: perform **G0 only**. Do not write a custom policy or generate hundreds of training scenes until the viewer, saved screenshot, OneJev image decision and local environment versions are verified. Then complete the **scripted physical baseline** before introducing model-controlled skill selection.